

5 - Visualization and Plotting

May 19, 2026

Trực quan hóa và biểu đồ

TS. Lê Thành Trung

May 19, 2026

Contents

1	Mục tiêu bài học	3
2	Trong 2 chiều	3
2.1	Quy trình vẽ biểu đồ hiệu quả	3
2.2	Các ví dụ cơ bản	3
3	Trong 3 chiều	8
3.1	Khi nào dùng biểu đồ 3D	8
3.2	Lưu ý khi vẽ và đọc biểu đồ 3D	8
3.3	Các ví dụ cơ bản	8
4	Tổng kết	16
5	Bài tập	17

1 Mục tiêu bài học

- Hiểu vai trò của trực quan hóa dữ liệu trong phân tích.
- Vẽ và tùy biến các biểu đồ 2D cơ bản bằng Matplotlib.
- Làm quen với biểu đồ 3D và các lưu ý khi diễn giải.

2 Trong 2 chiều

2.1 Quy trình vẽ biểu đồ hiệu quả

1. Xác định câu hỏi cần trả lời bằng biểu đồ.
2. Chọn đúng loại biểu đồ cho dữ liệu:
 - So sánh giá trị giữa các nhóm: bar chart.
 - Quan sát xu hướng theo thời gian: line chart.
 - Quan sát phân bố dữ liệu: histogram.
 - Kiểm tra mối quan hệ giữa 2 biến: scatter plot.
3. Đặt tiêu đề, nhãn trục, chú thích rõ ràng.
4. Tránh lạm dụng màu sắc và hiệu ứng.

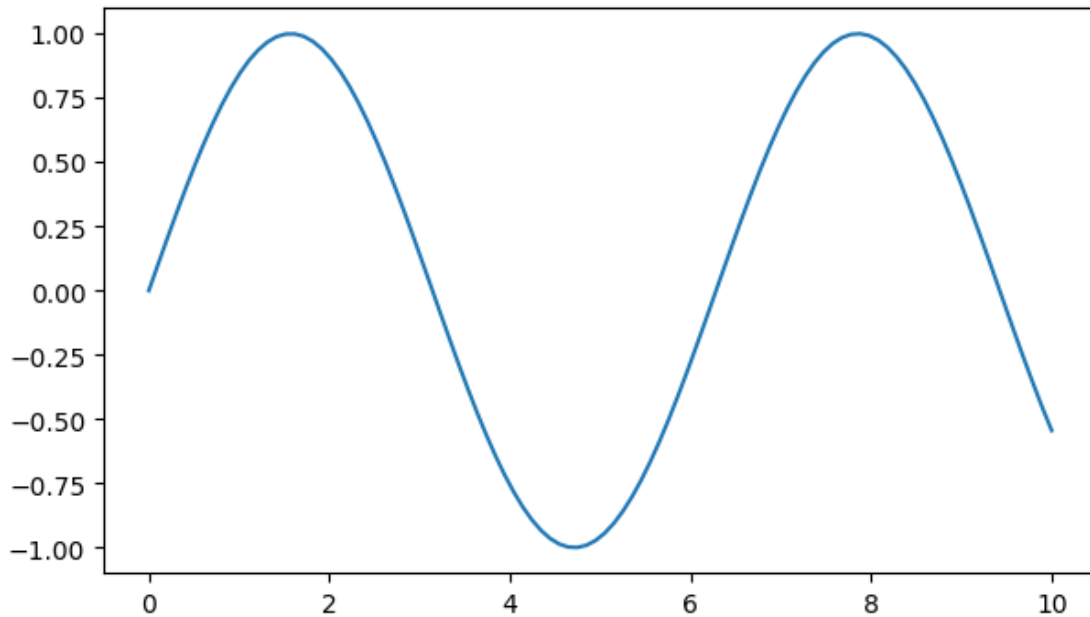
2.2 Các ví dụ cơ bản

- Vẽ biểu đồ đường cơ bản

```
[1]: # Mục 1 - Đơn giản nhất: vẽ 1 line chart cơ bản
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.figure(figsize=(7, 4))
plt.plot(x, y)
plt.show()
```

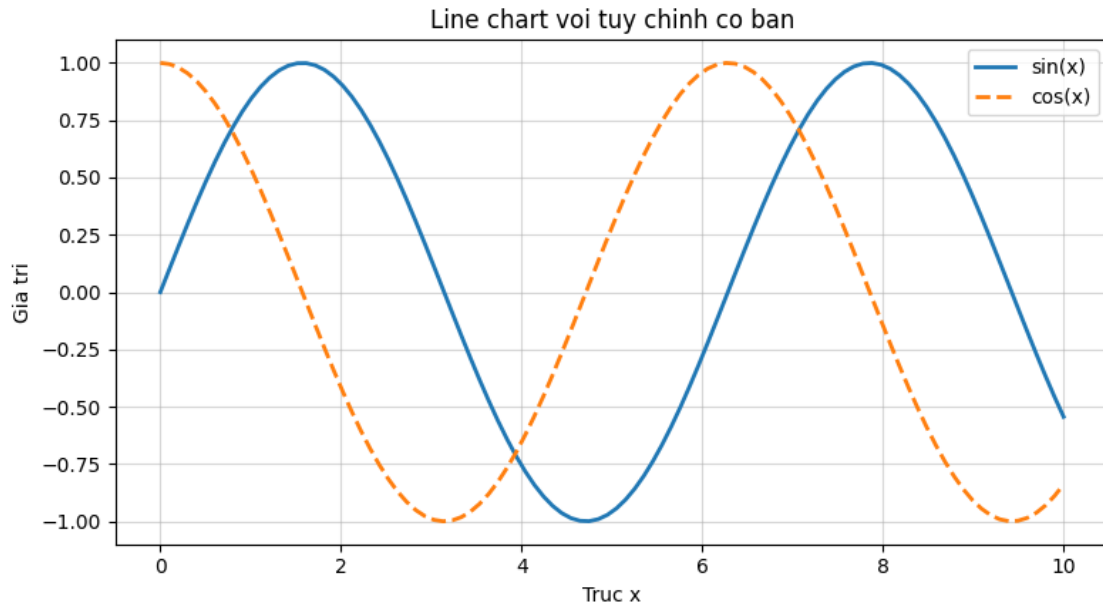


- Thêm các tùy chỉnh cơ bản

```
[8]: # Mục 2 - Thêm các tùy chỉnh cơ bản
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.figure(figsize=(8, 4.5))
plt.plot(x, y1, label="sin(x)", color="#1f77b4", linewidth=2)
plt.plot(x, y2, label="cos(x)", color="#ff7f0e", linewidth=2, linestyle="--")
plt.title("Line chart với tùy chỉnh cơ bản")
plt.xlabel("Trục x")
plt.ylabel("Giá trị")
plt.grid(alpha=0.5) # Thêm grid với độ mờ alpha=0.5, để tăng độ rõ ràng của
    ↪ biểu đồ
plt.legend()
plt.tight_layout() # Tự động điều chỉnh layout để tránh bị cắt nội dung
plt.show()
```



- Kết hợp nhiều loại biểu đồ trong cùng một hình

```
[11]: # Muc 3 - Ket hop nhieu loai bieu do trong cung mot figure
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

x = np.linspace(0, 10, 100)
axes[0, 0].plot(x, np.sin(x), label="sin(x)", linewidth=2)
axes[0, 0].plot(x, np.cos(x), label="cos(x)", linewidth=2)
axes[0, 0].set_title("Line")
axes[0, 0].legend()
axes[0, 0].grid(alpha=0.3)

sx = np.random.normal(0, 1, 120)
sy = 0.6 * sx + np.random.normal(0, 0.5, 120)
axes[0, 1].scatter(sx, sy, alpha=0.75, color="tab:orange")
axes[0, 1].set_title("Scatter")
axes[0, 1].grid(alpha=0.3)

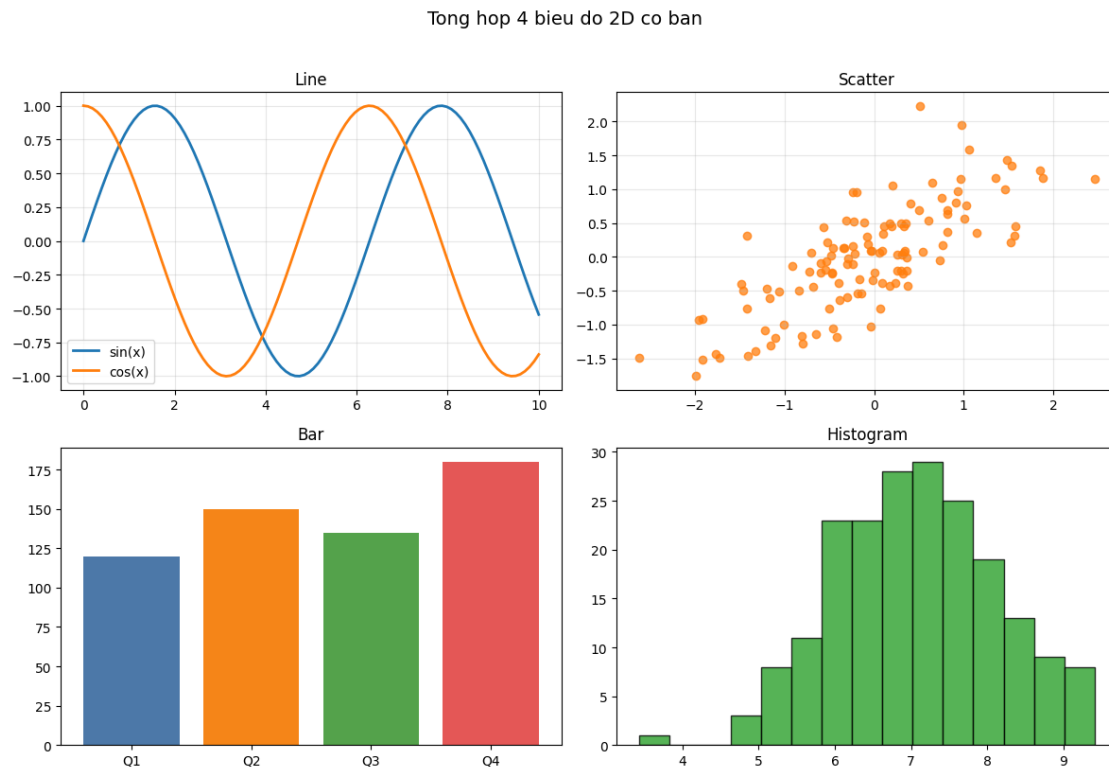
labels = ["Q1", "Q2", "Q3", "Q4"]
values = [120, 150, 135, 180]
axes[1, 0].bar(labels, values, color=["#4c78a8", "#f58518", "#54a24b", "#e45756"])
axes[1, 0].set_title("Bar")
```

```

scores = np.random.normal(7.0, 1.1, 200)
axes[1, 1].hist(scores, bins=15, color="tab:green", edgecolor="black", alpha=0.
↪8)
axes[1, 1].set_title("Histogram")

plt.suptitle("Tong hop 4 bieu do 2D co ban", fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

```



- Tùy chỉnh phức tạp hơn: style, annotation và bố cục dashboard

```

[16]: # Mục 4 - Tùy chỉnh phức tạp hơn: style, annotation, và bố cục dashboard
import numpy as np
import matplotlib.pyplot as plt

plt.style.use("seaborn-v0_8-whitegrid")
np.random.seed(10)

months = np.arange(1, 13)
revenue = np.array([20, 22, 19, 25, 27, 30, 34, 33, 31, 35, 37, 40])
cost = np.array([15, 16, 15, 18, 20, 22, 24, 24, 23, 25, 26, 28])
profit = revenue - cost

```

```

fig = plt.figure(figsize=(12, 7))
grid = fig.add_gridspec(2, 3, hspace=0.35, wspace=0.25)

ax1 = fig.add_subplot(grid[0, :2])
ax1.plot(months, revenue, marker="o", linewidth=2.5, label="Revenue",
        color="#2a9d8f")
ax1.plot(months, cost, marker="s", linewidth=2.0, label="Cost", color="#e76f51")
ax1.fill_between(months, cost, revenue, where=(revenue >= cost), alpha=0.2,
        color="#2a9d8f", label="Profit zone")
peak_idx = int(np.argmax(revenue))
ax1.annotate(
    "Peak revenue",
    xy=(months[peak_idx], revenue[peak_idx]),
    xytext=(months[peak_idx] - 2.2, revenue[peak_idx] + 2.5),
    arrowprops=dict(arrowstyle="->", color="black"),
    fontsize=9,
)
ax1.set_title("Doanh thu va chi phi theo thang")
ax1.set_xlabel("Thang")
ax1.set_ylabel("Gia tri")
ax1.legend(loc="upper left")

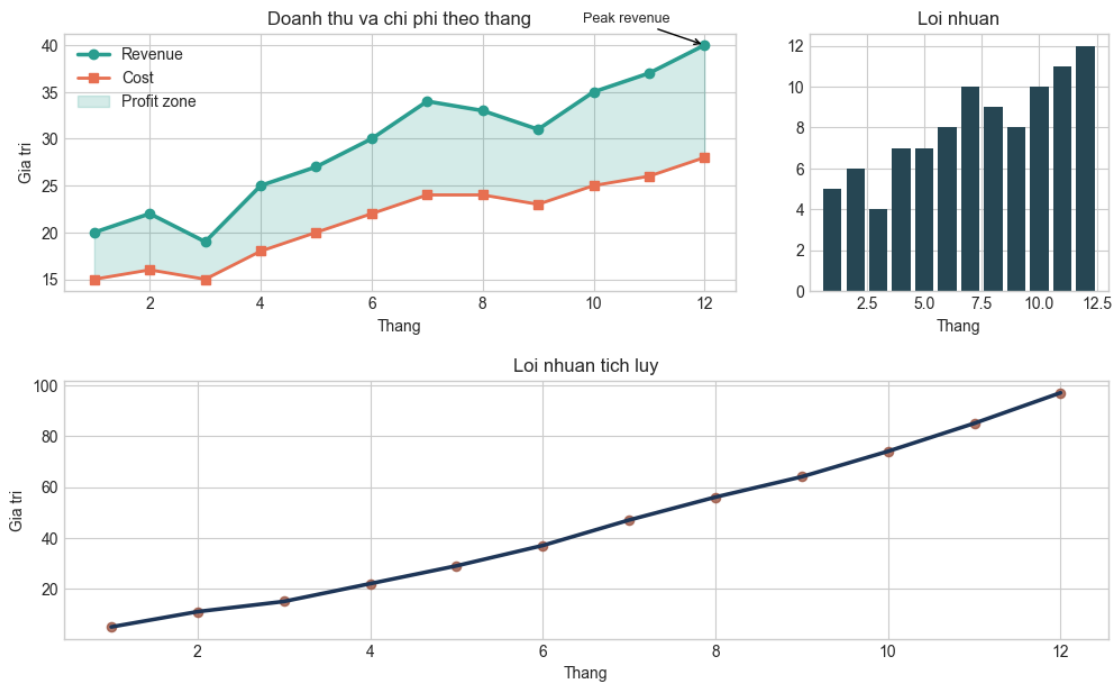
ax2 = fig.add_subplot(grid[0, 2])
ax2.bar(months, profit, color=np.where(profit >= 0, "#264653", "#d62828"))
ax2.set_title("Loi nhuan")
ax2.set_xlabel("Thang")

ax3 = fig.add_subplot(grid[1, :])
ax3.plot(months, np.cumsum(profit), color="#1d3557", linewidth=2.5)
ax3.scatter(months, np.cumsum(profit), color="#9d5745ce", s=35)
ax3.set_title("Loi nhuan tich luy")
ax3.set_xlabel("Thang")
ax3.set_ylabel("Gia tri")

fig.suptitle("Dashboard mini: tu line chart co ban den trinh bay chuyen
        nghiep", fontsize=14, fontweight="bold")
plt.show()

```

Dashboard mini: tu line chart cơ bản đến trình bày chuyên nghiệp



3 Trong 3 chiều

3.1 Khi nào dùng biểu đồ 3D

- Dữ liệu có 3 biến số liên tục cần quan sát cùng lúc.
- Muốn mô tả bề mặt hàm số $z = f(x, y)$.

3.2 Lưu ý khi vẽ và đọc biểu đồ 3D

- Góc nhìn có thể làm sai lệch cảm nhận khoảng cách.
- Nên kết hợp thêm biểu đồ 2D nếu cần so sánh chi tiết.
- Luôn ghi rõ tên trục x , y , z và đơn vị đo.

3.3 Các ví dụ cơ bản

- Ví dụ 1

```
[25]: # Ví dụ 1: kết hợp 3D Scatter và 3D Surface trong cùng một figure
# 3D Scatter
np.random.seed(7)
n = 150
x3 = np.random.uniform(-3, 3, n)
y3 = np.random.uniform(-3, 3, n)
z3 = 0.5 * x3**2 - 0.3 * y3 + np.random.normal(0, 0.7, n)
```



```

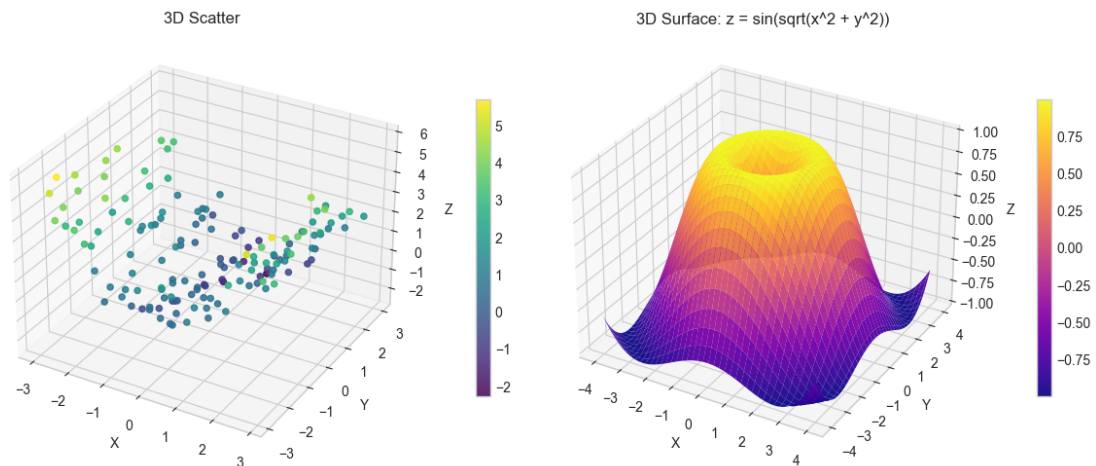
fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
sc = ax1.scatter(x3, y3, z3, c=z3, cmap="viridis", alpha=0.85)
ax1.set_title("3D Scatter")
ax1.set_xlabel("X")
ax1.set_ylabel("Y")
ax1.set_zlabel("Z")
fig.colorbar(sc, ax=ax1, shrink=0.65, pad=0.1)

# 3D Surface
x = np.linspace(-4, 4, 80)
y = np.linspace(-4, 4, 80)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

ax2 = fig.add_subplot(1, 2, 2, projection="3d")
surf = ax2.plot_surface(X, Y, Z, cmap="plasma", linewidth=0, antialiased=True,
    alpha=0.95)
ax2.set_title("3D Surface: z = sin(sqrt(x^2 + y^2))")
ax2.set_xlabel("X")
ax2.set_ylabel("Y")
ax2.set_zlabel("Z")
fig.colorbar(surf, ax=ax2, shrink=0.65, pad=0.1)

plt.tight_layout()
plt.show()

```



- Ví dụ 2

```
[26]: # Ví dụ 2: 3D Line Chart - Đường cong trong không gian 3D
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(10, 5))

# Hình xoắn ốc 3D (Helix)
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
t = np.linspace(0, 4 * np.pi, 100)
x = np.cos(t)
y = np.sin(t)
z = t

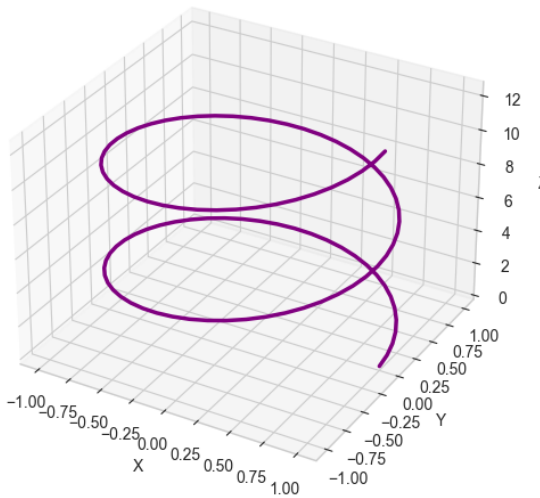
ax1.plot(x, y, z, linewidth=2.5, color="purple")
ax1.set_title("Hình xoắn ốc 3D (Helix)")
ax1.set_xlabel("X")
ax1.set_ylabel("Y")
ax1.set_zlabel("Z")

# Đường cong lượng giác 3D
ax2 = fig.add_subplot(1, 2, 2, projection="3d")
t2 = np.linspace(0, 10 * np.pi, 150)
x2 = t2 * np.cos(t2)
y2 = t2 * np.sin(t2)
z2 = t2

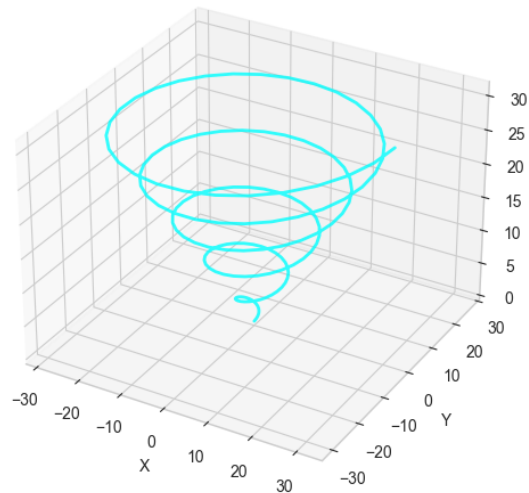
ax2.plot(x2, y2, z2, linewidth=2, color="cyan", alpha=0.8)
ax2.set_title("Đường xoắn mở rộng 3D")
ax2.set_xlabel("X")
ax2.set_ylabel("Y")
ax2.set_zlabel("Z")

plt.tight_layout()
plt.show()
```

Hình xoắn ốc 3D (Helix)



Đường xoắn mở rộng 3D



- Ví dụ 3

```
[27]: # Ví dụ 3: 3D Bar Chart - Biểu đồ cột trong không gian 3D
fig = plt.figure(figsize=(10, 5))

# 3D Bar Chart đơn giản
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
xpos = np.arange(4)
ypos = np.arange(3)
zpos = np.zeros(12)

# Tạo dữ liệu: ma trận 3x4
data = np.random.randint(5, 20, size=(3, 4))
colors = ["red", "blue", "green"]
dx = dy = 0.8
dz = data.flatten()

for i in range(3):
    for j in range(4):
        ax1.bar3d(j, i, 0, dx, dy, data[i, j], color=colors[i], alpha=0.7)

ax1.set_title("3D Bar Chart")
ax1.set_xlabel("X")
ax1.set_ylabel("Y")
ax1.set_zlabel("Giá trị")

# 3D Stem Plot (plot kiểu que diêm)
ax2 = fig.add_subplot(1, 2, 2, projection="3d")
```

```

theta = np.linspace(0, 2 * np.pi, 12)
z_vals = np.sin(theta)
x_vals = np.cos(theta)
y_vals = np.ones_like(theta)

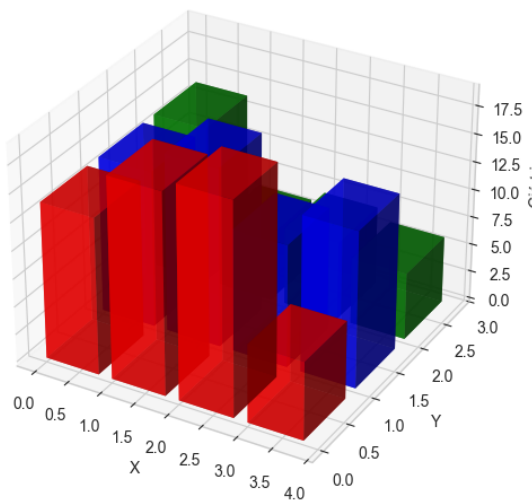
colors_stem = plt.cm.viridis(np.linspace(0, 1, len(theta)))
for i in range(len(theta)):
    ax2.plot([x_vals[i], x_vals[i]], [y_vals[i], y_vals[i]], [0, z_vals[i]],
            color=colors_stem[i], linewidth=2)
    ax2.scatter(x_vals[i], y_vals[i], z_vals[i], color=colors_stem[i], s=50)

ax2.set_title("3D Stem Plot")
ax2.set_xlabel("X")
ax2.set_ylabel("Y")
ax2.set_zlabel("Z")

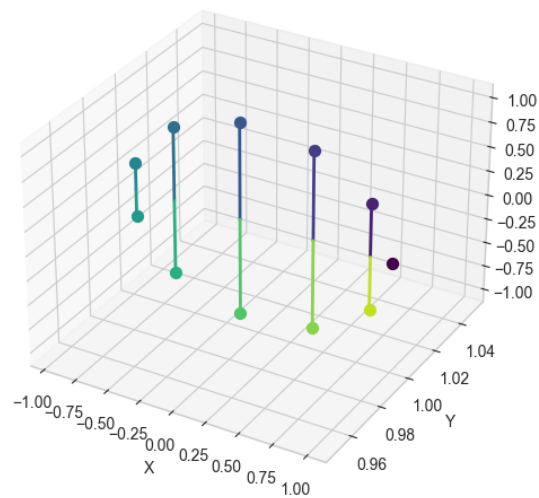
plt.tight_layout()
plt.show()

```

3D Bar Chart



3D Stem Plot



- Ví dụ 4

[28]: *# Ví dụ 4: 3D Wireframe và Contour - Tìm hiểu cấu trúc bề mặt*

```

fig = plt.figure(figsize=(12, 5))

# Tạo dữ liệu hàm 3D: z = x^2 + y^2
x = np.linspace(-3, 3, 40)
y = np.linspace(-3, 3, 40)
X, Y = np.meshgrid(x, y)

```

```

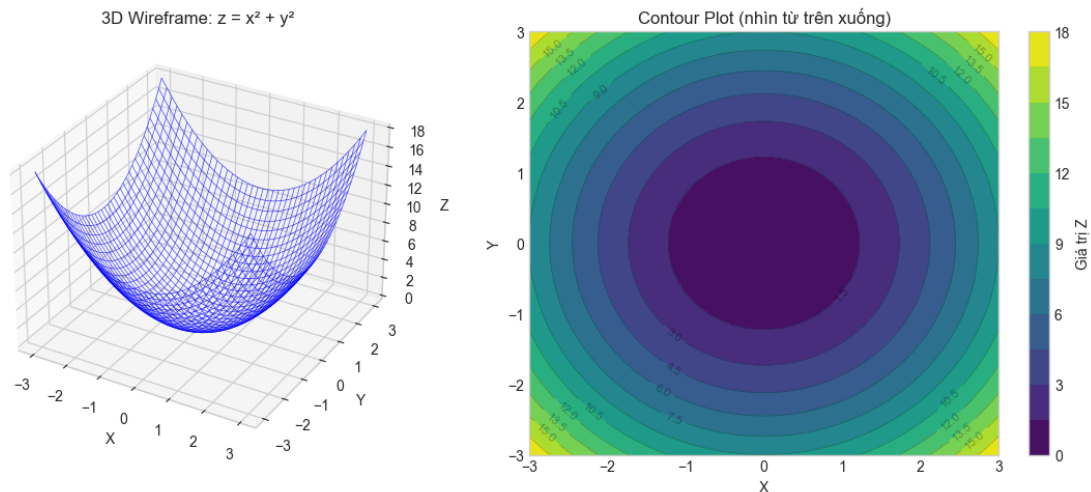
Z_paraboloid = X**2 + Y**2

# Wireframe: chỉ vẽ khung, không tô màu
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.plot_wireframe(X, Y, Z_paraboloid, color="blue", linewidth=0.5, alpha=0.7)
ax1.set_title("3D Wireframe:  $z = x^2 + y^2$ ")
ax1.set_xlabel("X")
ax1.set_ylabel("Y")
ax1.set_zlabel("Z")

# Contour Plot: chiếu bề mặt xuống mặt phẳng 2D
ax2 = fig.add_subplot(1, 2, 2)
contour = ax2.contourf(X, Y, Z_paraboloid, levels=15, cmap="viridis")
contour_lines = ax2.contour(X, Y, Z_paraboloid, levels=15, colors="black",
    ↳linewidths=0.5, alpha=0.3)
ax2.clabel(contour_lines, inline=True, fontsize=8)
plt.colorbar(contour, ax=ax2, label="Giá trị Z")
ax2.set_title("Contour Plot (nhìn từ trên xuống)")
ax2.set_xlabel("X")
ax2.set_ylabel("Y")

plt.tight_layout()
plt.show()

```



- Ví dụ 5

[29]:

```

# Ví dụ 5: So sánh nhiều hàm 3D - từ cơ bản đến phức tạp
fig = plt.figure(figsize=(14, 5))

x = np.linspace(-3, 3, 50)

```

```

y = np.linspace(-3, 3, 50)
X, Y = np.meshgrid(x, y)

# Hàm 1: Paraboloid
Z1 = X**2 + Y**2

# Hàm 2: Saddle (yên ngựa)
Z2 = X**2 - Y**2

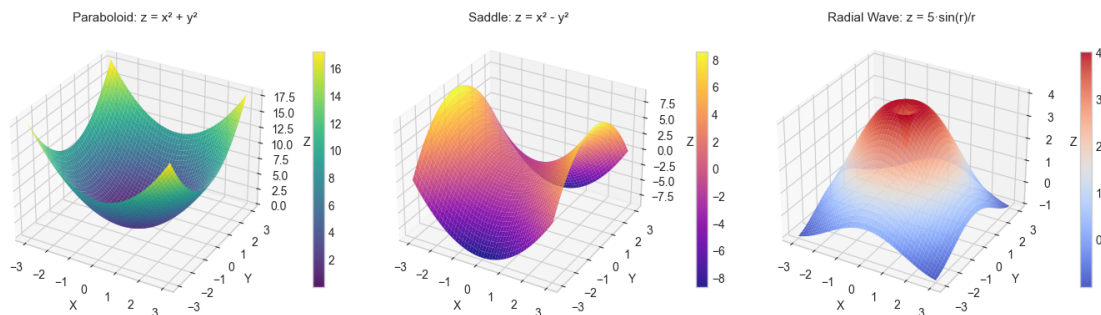
# Hàm 3: Sóng sin tích hợp
Z3 = 5 * np.sin(np.sqrt(X**2 + Y**2)) / (np.sqrt(X**2 + Y**2) + 0.1)

# Vẽ 3 bề mặt
cmaps = ["viridis", "plasma", "coolwarm"]
titles = ["Paraboloid:  $z = x^2 + y^2$ ", "Saddle:  $z = x^2 - y^2$ ", "Radial Wave:  $z = \frac{5 \cdot \sin(r)}{r}$ "]

for idx, (Z, cmap, title) in enumerate(zip([Z1, Z2, Z3], cmaps, titles)):
    ax = fig.add_subplot(1, 3, idx + 1, projection="3d")
    surf = ax.plot_surface(X, Y, Z, cmap=cmap, alpha=0.9, edgecolor="none")
    ax.set_title(title, fontsize=11)
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.set_zlabel("Z")
    fig.colorbar(surf, ax=ax, shrink=0.6, pad=0.1)

plt.tight_layout()
plt.show()

```



- Ví dụ 6

```

[30]: # Ví dụ 6: Kết hợp nhiều loại plot và góc nhìn khác nhau
fig = plt.figure(figsize=(14, 10))

x = np.linspace(-4, 4, 50)

```

```

y = np.linspace(-4, 4, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2)) * np.exp(-0.1 * (X**2 + Y**2))

# Plot 1: Surface với view từ trên
ax1 = fig.add_subplot(2, 3, 1, projection="3d")
surf1 = ax1.plot_surface(X, Y, Z, cmap="twilight", alpha=0.8)
ax1.view_init(elev=70, azimuth=45)
ax1.set_title("Góc nhìn cao (elev=70°)")
ax1.set_zlim(-0.5, 0.5)

# Plot 2: Surface với view từ chính diện
ax2 = fig.add_subplot(2, 3, 2, projection="3d")
surf2 = ax2.plot_surface(X, Y, Z, cmap="twilight", alpha=0.8)
ax2.view_init(elev=10, azimuth=0)
ax2.set_title("Góc nhìn chính diện (elev=10°)")
ax2.set_zlim(-0.5, 0.5)

# Plot 3: Surface với view từ bên
ax3 = fig.add_subplot(2, 3, 3, projection="3d")
surf3 = ax3.plot_surface(X, Y, Z, cmap="twilight", alpha=0.8)
ax3.view_init(elev=0, azimuth=90)
ax3.set_title("Góc nhìn từ bên (elev=0°)")
ax3.set_zlim(-0.5, 0.5)

# Plot 4: Wireframe + Scatter
ax4 = fig.add_subplot(2, 3, 4, projection="3d")
ax4.plot_wireframe(X, Y, Z, color="blue", linewidth=0.3, alpha=0.3)
# Thêm scatter tại các điểm đặc biệt
indices = np.random.choice(X.size, size=50, replace=False)
x_scatter = X.flat[indices]
y_scatter = Y.flat[indices]
z_scatter = Z.flat[indices]
ax4.scatter(x_scatter, y_scatter, z_scatter, c=z_scatter, cmap="coolwarm",
            s=30, alpha=0.8)
ax4.set_title("Wireframe + Scatter overlay")

# Plot 5: Contour + 3D Surface
ax5 = fig.add_subplot(2, 3, 5)
contourf = ax5.contourf(X, Y, Z, levels=20, cmap="twilight")
ax5.contour(X, Y, Z, levels=10, colors="black", linewidths=0.5, alpha=0.2)
plt.colorbar(contourf, ax=ax5)
ax5.set_title("Contour plot chi tiết")
ax5.set_xlabel("X")
ax5.set_ylabel("Y")

# Plot 6: Thống kê Z

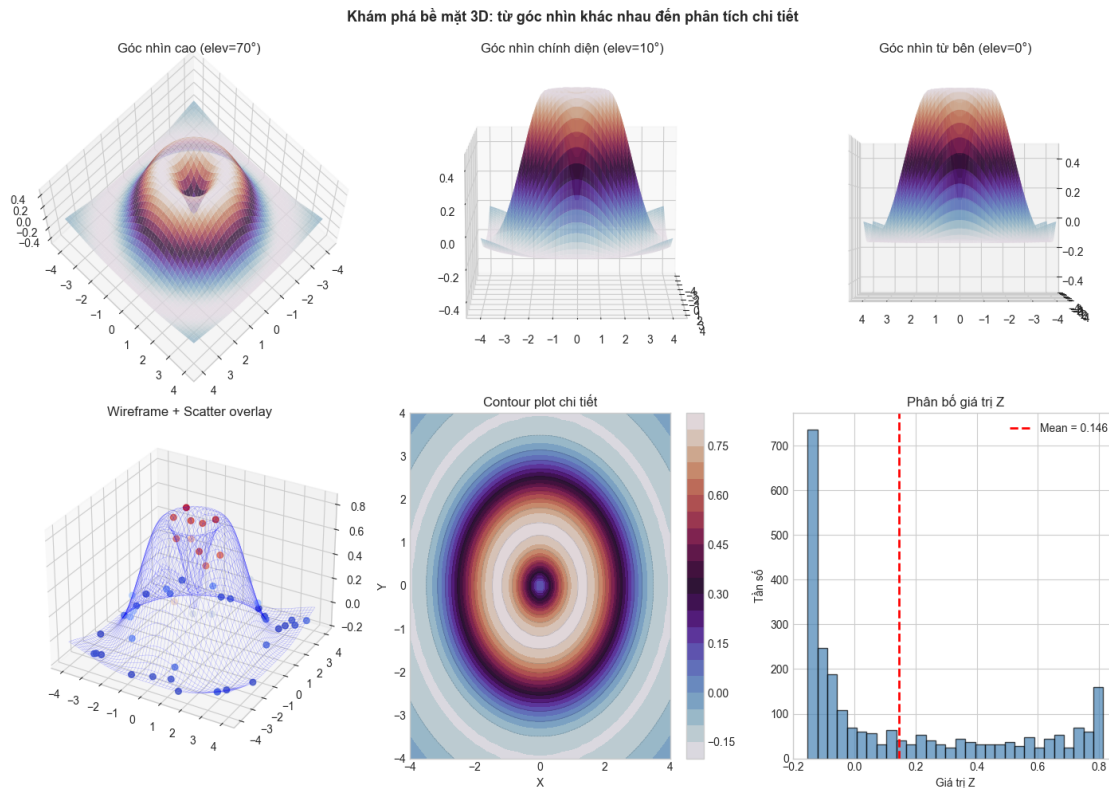
```

```

ax6 = fig.add_subplot(2, 3, 6)
z_flat = Z.flatten()
ax6.hist(z_flat, bins=30, color="steelblue", edgecolor="black", alpha=0.7)
ax6.axvline(z_flat.mean(), color="red", linestyle="--", linewidth=2,
            label=f"Mean = {z_flat.mean():.3f}")
ax6.set_title("Phân bố giá trị Z")
ax6.set_xlabel("Giá trị Z")
ax6.set_ylabel("Tần số")
ax6.legend()

plt.suptitle("Khám phá bề mặt 3D: từ góc nhìn khác nhau đến phân tích chi tiết",
             fontsize=13, fontweight="bold", y=0.995)
plt.tight_layout()
plt.show()

```



4 Tổng kết

- Biểu đồ 2D phù hợp cho đa số bài toán phân tích cơ bản.
- Biểu đồ 3D hữu ích khi cần biểu diễn thêm chiều dữ liệu, nhưng cần cẩn trọng khi diễn giải.
- Một biểu đồ tốt luôn có: tiêu đề, nhãn trục, thang đo rõ ràng, và màu sắc hợp lý.

5 Bài tập

1. Tạo bộ dữ liệu bất kỳ và vẽ đủ 4 biểu đồ 2D đã học.
2. Vẽ biểu đồ 3D cho hàm: $z = x^2 - y^2$.
3. Chọn một biểu đồ trong bài và tùy biến theme (màu, font, grid, legend) để dễ đọc hơn.